

React Native

Learning Field Guide

Gautam's Personal Code Walkthrough

Windows · Expo SDK 54 · Android

Every file in the BeReal clone codebase — walked through line by line.
Every decision explained. Every bug annotated in context.
Read Part 1 (Concept Glossary) before this if you haven't already.

#F01	lib/supabase/client.ts	Supabase client setup + AsyncStorage session
#F02	context/AuthContext.tsx	Auth state, signIn, signUp — the brain
#F03	app/_layout.tsx	Root layout, auth guard, two-function pattern
#F04	app/(auth)/_layout.tsx	Stack navigator for auth flow
#F05	app/(auth)/login.tsx	Login screen — 4 active bugs found
#F06	app/(auth)/signup.tsx	Signup screen — zombie useEffect bug
#F07	app/(auth)/onboarding.tsx	Image picker + Supabase — typo bug
#F08	app/(tabs)/_layout.tsx	Tab navigator + Ionicons
#F09	app/(tabs)/profile.tsx	useRef, useCallback, BottomSheet
#F10	app/(tabs)/index.tsx	Home screen playground

What's In Part 2

This is the practical chapter. Every file from your actual codebase, walked through line by line. Every import explained. Every function broken down. Every bug annotated where it appears. Use Part 1 as the theory reference if a concept needs refreshing.

F01 `lib/supabase/client.ts`

Supabase client, AsyncStorage, env vars

F02 `context/AuthContext.tsx`

createContext, useState, signIn, signUp — full deep dive

F03 `app/_layout.tsx`

Root layout, auth guard, two-function pattern

F04 `app/(auth)/_layout.tsx`

Stack navigator for login/signup/onboarding

F05 `app/(auth)/login.tsx`

Login UI — 4 bugs found in your file

F06 `app/(auth)/signup.tsx`

Signup form — zombie useEffect + password bug

F07 `app/(auth)/onboarding.tsx`

Image picker, camera, Supabase query + typo bug

F08 `app/(tabs)/_layout.tsx`

Tab navigator, Ionicons, focused prop

F09 `app/(tabs)/profile.tsx`

useRef, useCallback, BottomSheet

F10 `app/(tabs)/index.tsx`

Home screen — early playground code

client.ts

lib/supabase/client.ts

Purpose of this file

This file creates a single configured Supabase client that every other file imports. You create it once here and reuse it everywhere — instead of configuring Supabase in every file that needs it. It also wires up AsyncStorage so that login sessions survive app restarts.

```
// lib/supabase/client.ts – full file annotated
import "react-native-url-polyfill/auto";
// Supabase uses the URL class internally.
// React Native does not have a built-in URL implementation.
// This polyfill adds it. Must be imported before Supabase.
import AsyncStorage from "@react-native-async-storage/async-storage";
// AsyncStorage = key-value storage that persists across app restarts.
// Like localStorage for mobile. Supabase uses it to save the
// auth token so you stay logged in when you close and reopen the app.
import { createClient } from "@supabase/supabase-js";
// createClient = the Supabase factory function.
// Pass it your project URL + anon key and it gives you a client.
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!;
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!;
// process.env = reads from your .env file.
// EXPO_PUBLIC_ prefix = Expo will expose this to the JS bundle.
// Without EXPO_PUBLIC_, the value is undefined at runtime.
// The ! at the end = TypeScript "trust me, this is not undefined".
// (non-null assertion – use carefully)
export const supabase = createClient(supabaseUrl, supabaseKey, {
  auth: {
    storage: AsyncStorage,
    // Use AsyncStorage instead of the default (memory-only).
    // Without this: logging in works but session is gone on restart.
    autoRefreshToken: true,
    // Automatically refresh the JWT token before it expires.
    // Without this: users get logged out after 1 hour.
    persistSession: true,
    // Save session to AsyncStorage so it survives app restarts.
    detectSessionInUrl: false,
    // For OAuth deep links. false = not using OAuth, no URL parsing.
  },
});
```

Why export const supabase — not export default

Named exports (`export const`) let you export multiple things from one file. Default exports (`export default`) only allow one. Convention in utility files: use named exports. That is why you write: `import { supabase } from '../lib/supabase/client'`.

AuthContext.tsx

context/AuthContext.tsx

INCOMPLETE

Purpose of this file

This is the brain of your entire auth system. It creates a React Context that holds the logged-in user object and exposes `signIn` and `signUp` functions to every screen that needs them — without passing them as props through every layout and navigator.

```
// AuthContext.tsx – imports
import React, { createContext, useState, useContext, ReactNode }
from "react";
// createContext = creates the Context object (the data pipe)
// useState = manages user state inside AuthProvider
// useContext = lets any component READ from the context
// ReactNode = TypeScript type: "anything React can render"
// used to type the children prop
import { supabase } from "../lib/supabase/client";
// Your configured Supabase client from F01.
// The ../ goes UP one folder from context/ to project root,
// then into lib/supabase/client.ts.
```

```
// AuthContext.tsx – interfaces
export interface User {
  id: string; // Supabase UUID
  name: string; // display name from profiles table
  email: string;
  username: string;
  profilePicture?: string; // ? = optional, may not exist yet
  onboardingCompleted: boolean;
}
// interface = TypeScript contract describing object shape.
// If you try to use a User object missing any required field,
// TypeScript throws a red underline immediately.
interface AuthContextType {
  user: User | null;
  // user = User object when logged in, null when logged out.
  // The | is a union type: "this type OR that type".
  signUp: (email: string, password: string) => Promise<void>;
  signIn: (email: string, password: string) => Promise<void>;
  // Promise<void> = async function that returns nothing useful.
  // THIS is why isAuth caused an error.
  // AuthContextType has: user, signUp, signIn.
  // No isAuth. TypeScript caught it before runtime.
}
```

```
// AuthContext.tsx – createContext and AuthProvider
const AuthContext = createContext<AuthContextType | undefined>(undefined);
// Creates the Context object – the pipe.
// <AuthContextType | undefined> = TypeScript generic.
// The context holds AuthContextType OR undefined.
// undefined = what you get if no Provider is above you in the tree.
// This line is at module level (outside any function) – created once.
export const AuthProvider = ({ children }: { children: ReactNode }) => {
// AuthProvider is a component that wraps your app tree.
// children = everything rendered inside <AuthProvider>...</AuthProvider>
// { children }: { children: ReactNode } = destructured typed prop.
const [user, setUser] = useState<User | null>(null);
// user starts as null = nobody logged in.
// <User | null> = TypeScript generic for useState.
// setUser() will be called after successful signIn/signUp.
return (
<AuthContext.Provider value={{ user, signUp, signIn }}>
  {children}
</AuthContext.Provider>
// value = what every consumer receives from useAuth().
// Must match AuthContextType exactly.
);
};
```

```
// AuthContext.tsx – signIn function
const signIn = async (email: string, password: string) => {
const { data, error } = await supabase.auth.signInWithPassword(
  { email, password }
);
// Supabase ALWAYS returns { data, error }.
// data.user = the authenticated Supabase User object on success
// error = Error object on failure, null on success
if (error) throw error;
// throw sends the error to the catch() in handleSignIn.
// handleSignIn then shows Alert.alert() to the user.
if (data.user) {
console.log("Signed in:", data.user);
// BUG: setUser() is NOT called here.
// user state stays null even after successful login.
// _layout.tsx auth guard sees null = redirects to login forever.
// FIX (next tutorial step): map data.user to your User interface
// and call setUser() here.
}
};
```

Known gap: setUser() is never called — auth guard loops forever

After signIn() or signUp() succeeds, the user state in AuthContext stays null. The auth guard in _layout.tsx checks user — sees null — redirects to login. Even though login worked, the app acts like it didn't. Next tutorial step: after signIn, map data.user to your User interface and call setUser() to update the state.

```
// AuthContext.tsx – useAuth hook
export const useAuth = () => {
  const context = useContext(AuthContext);
  // useContext searches UP the component tree for the nearest
  // AuthContext.Provider and returns its value prop.
  if (context === undefined) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  // Safety check: if context is undefined, useAuth() was called
  // without an AuthProvider above it in the tree.
  // The error message tells you exactly what went wrong.
  return context; // returns { user, signIn, signUp }
};

// Usage in any screen (after AuthProvider wraps the tree):
const { signIn } = useAuth(); // login.tsx
const { signUp } = useAuth(); // signup.tsx
const { user } = useAuth(); // _layout.tsx RootLayoutNav
```

_layout.tsx

app/_layout.tsx (ROOT)

HAS BUGS

Purpose of this file

The root layout is the entry point of your entire app. Every screen renders inside it. It has two jobs: wrap the app in AuthProvider (so all screens can access auth state), and redirect the user to the correct screen on startup (login if not authenticated, home tabs if authenticated).

```
// app/_layout.tsx – imports annotated
import { AuthProvider } from "@context/AuthContext";
// @/ = path alias pointing to project root.
// Without it: "../../context/AuthContext" from nested files.
// @/ never breaks when you move files around.
import { Stack, useRouter, useRootNavigationState }
from "expo-router";
// Stack = navigator: screens slide in from right
// useRouter = gives you push(), replace(), back()
// useRootNavigationState = check if Expo Router is ready to navigate
import { GestureHandlerRootView }
from "react-native-gesture-handler";
// Required ancestor for ANY gesture-based component.
// BottomSheet, swipeable lists, drag-and-drop all need this.
// If missing: silent crash on first gesture interaction.
```

```
// app/_layout.tsx – the two-function pattern
export default function RootLayout() {
  // This function RENDERS the Provider.
  // CANNOT call useAuth() here – Provider not mounted yet.
  return (
    <AuthProvider>
    <GestureHandlerRootView style={{ flex: 1 }}>
    <Stack screenOptions={{ headerShown: false }}>
    <Stack.Screen name="(tabs)" />
    <Stack.Screen name="(auth)" />
    </Stack>
    </GestureHandlerRootView>
    </AuthProvider>
  );
}
// NOTE: your uploaded file wraps Stack inside RootLayoutNav.
// The pattern above is the cleaner final version.
```



```
// app/_layout.tsx – auth guard with useEffect
// In your file this logic lives inside RootLayout directly.
// The navigation guard:
const router = useRouter();
const navigationState = useRootNavigationState();
const isAuthenticated = false;
// BUG: isAuthenticated is hardcoded to false!
// Should be: const { user } = useAuth();
// Then use: if (!user) instead of if (!isAuthenticated)
useEffect(() => {
  if (!navigationState?.key) return;
  // navigationState?.key = optional chaining.
  // If navigationState is null/undefined -> returns undefined -> stops.
  // This guard prevents navigating before Expo Router is ready.
  // Without it: "Attempted to navigate before mounting" crash.
  const timeout = setTimeout(() => {
    if (!isAuthenticated) router.replace("/(auth)/login");
    else router.replace("/(tabs)");
  }, 0);
  // setTimeout(fn, 0) = defer to next event loop tick.
  // Ensures Expo Router is fully mounted before navigation.
  return () => clearTimeout(timeout);
  // Cleanup: cancel if component unmounts before timeout fires.
}, []);
```

Active bug: isAuthenticated = false is hardcoded

With isAuthenticated always false, the app always redirects to login — even after a successful signup or login. Fix: replace `const isAuthenticated = false` with `const { user } = useAuth()` and change `if (!isAuthenticated)` to `if (!user)`. But this only fully works AFTER `setUser()` is implemented in `AuthContext`.

`_layout.tsx`

`app/(auth)/_layout.tsx`

Purpose of this file

This layout defines how the auth screens (login, signup, onboarding) are presented. It uses a Stack navigator, meaning each screen slides in from the right and the user can swipe back. The header bar is hidden because you built your own navigation UI (the Sign In / Sign Up links at the bottom).

```
// app/(auth)/_layout.tsx – full file
import { Stack } from "expo-router";
export default function AuthLayout() {
  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Screen name="login" />
      <Stack.Screen name="signup" />
      // name matches the FILENAME without .tsx extension.
      // onboarding is NOT registered here – that is fine.
      // Unregistered screens still work with default Stack settings.
    </Stack>
  );
}
// Stack navigator behaviour:
// Screens slide in from the RIGHT when navigating forward.
// Back gesture / back button returns to previous screen.
// headerShown: false = removes the default navigation header bar.
```

Why (auth) has its own `_layout.tsx`

Each folder in `app/` can have its own navigator type. The root `_layout.tsx` uses Stack for the top-level app flow. This `auth/_layout.tsx` also uses Stack — but scoped just to auth screens. The `tabs/_layout.tsx` uses Tabs. Nested navigators is how complex app navigation is built.

login.tsx

app/(auth)/login.tsx

HAS BUGS

Purpose of this file

The login screen. Collects email and password, calls `signIn()` from `AuthContext`, and lets the auth guard in `_layout.tsx` handle navigation. Your uploaded version is an early draft with 4 active bugs.

4 active bugs found in your uploaded login.tsx

BUG 1: `import { useRoute } from '@react-navigation/native'` — imported but never used. Stale import from an earlier version. BUG 2: `TextInput` imported from `'react-native-gesture-handler'` instead of `'react-native'`. These are different components and the gesture-handler version behaves unexpectedly on Android. BUG 3: The Sign In `TouchableOpacity` has no `onPress` handler. Tapping does literally nothing. BUG 4: No `useState` for email or password. The form captures nothing the user types.

```
// login.tsx – correct complete version
import { useRouter } from "expo-router";
import { useState } from "react";
import {
  View, Text, TextInput, TouchableOpacity,
  StyleSheet, Alert, ActivityIndicator
} from "react-native";
// TextInput from react-native – NOT gesture-handler!
import { SafeAreaView } from "react-native-safe-area-context";
import { useAuth } from "../../context/AuthContext";
export default function LoginScreen() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const router = useRouter();
  const { signIn } = useAuth();
  const handleSignIn = async () => {
    if (!email || !password) {
      Alert.alert("Error", "Please enter both email and password");
      return; // early return = stop, do not proceed
    }
    setIsLoading(true);
    try {
      await signIn(email, password);
      // Navigation handled by auth guard in _layout.tsx
      // once setUser() is implemented in AuthContext.
    } catch (error: any) {
      Alert.alert("Error", error.message || "Sign in failed");
    } finally {
      setIsLoading(false); // always stop the spinner
    }
  };
  // JSX: button with conditional spinner
  // <TouchableOpacity onPress={handleSignIn}>
  // {isLoading
  // ? <ActivityIndicator size={24} color="#fff" />
  // : <Text>Sign In</Text>
  // }
  // </TouchableOpacity>
```

```
// login.tsx – navigation link to signup
<TouchableOpacity
  style={styles.linkButton}
  onPress={() => router.push("/(auth)/signup")}
>
  <Text style={styles.linkButtonText}>
    Not have any account?{" "}
    <Text style={styles.linkButtonTextBold}>Signup</Text>
  </Text>
</TouchableOpacity>
// {" "} = a literal space character in JSX.
// Without it, "account?" and "Signup" would be squished together.
// The nested <Text> inside <Text> applies a different style
// to just the "Signup" word – bold + black color.
```

signup.tsx

app/(auth)/signup.tsx

HAS BUGS

Purpose of this file

The signup screen. Collects email and password, validates them, calls `signUp()` from `AuthContext`, then navigates to onboarding. Your uploaded version has two active bugs.

2 active bugs in your uploaded signup.tsx

BUG 1 (line 22-24): `useEffect(() => { router.push('/(auth)/onboarding'); }, []).` Fires instantly on mount. Every person who opens signup gets teleported to onboarding before typing anything. Delete this entire `useEffect` block. BUG 2: password minimum is 3 characters. Supabase requires at least 6. Any signup with a 3-5 character password silently fails at the Supabase level.

```
// signup.tsx – correct handleSignUp
const handleSignUp = async () => {
  if (!email || !password) {
    Alert.alert("Error", "Please enter both email and password");
    return;
  }
  if (password.length < 6) {
    Alert.alert("Error", "Password must be at least 6 characters");
    return;
  }
  // Supabase minimum = 6 characters. Your file has 3. Fix this.
  setisLoading(true);
  try {
    await signUp(email, password);
    router.replace("/(auth)/onboarding");
    // replace not push:
    // push = user can swipe back to signup. Bad.
    // replace = cannot go back. Correct for after registration.
  } catch (error: any) {
    Alert.alert("Error", error.message || "Failed to sign up");
  } finally {
    setisLoading(false); // ALWAYS stop the spinner
  }
};
```

```
// signup.tsx – the ActivityIndicator pattern
<TouchableOpacity style={styles.button} onPress={handleSignUp}>
  {isLoading
    ? <ActivityIndicator size={24} color="#fff" />
    : <Text style={styles.buttonText}>Sign Up</Text>
  }
</TouchableOpacity>
// When isLoading is true: button shows spinner, user cannot re-tap.
// When isLoading is false: button shows text, user can tap.
// This pattern prevents double-submission.
// Without it: user taps twice -> two signUp() calls -> Supabase error.
```

onboarding.tsx

app/(auth)/onboarding.tsx

HAS BUGS

Purpose of this file

After signup, users complete their profile here: choose a display name, username, and profile picture. This screen introduces three new patterns: image picking from the media library, taking a photo with the camera, and querying Supabase to check if a username is already taken.

```
// onboarding.tsx – imports
import { supabase } from "@/lib/supabase/client";
import * as ImagePicker from "expo-image-picker";
// * as = import ALL named exports as one namespace object.
// So: ImagePicker.launchImageLibraryAsync(), etc.
// This is the common pattern for Expo modules.
import { useState } from "react";
import {
  ActivityIndicator, Alert, Image, StyleSheet,
  Text, TextInput, TouchableOpacity, View,
} from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
```

```
// onboarding.tsx – state variables
const [name, setName] = useState("");
const [username, setUsername] = useState("");
const [isLoading, setIsLoading] = useState(false);
const [profileImage, setProfileImage] = useState<string | null>(null);
// profileImage stores a LOCAL file URI:
// null = no image selected yet (shows placeholder "+" circle)
// "file:///storage/emulated/0/..." = image selected from gallery
// This URI is temporary – it will disappear if the app restarts.
// Uploading to Supabase Storage is the next tutorial step.
```



```
// onboarding.tsx – pickImage (media library)
const pickImage = async () => {
  const { status } =
    await ImagePicker.requestMediaLibraryPermissionsAsync();
  // Shows Android permission dialog on FIRST call only.
  // Android remembers the choice. status = "granted" or "denied".
  if (status !== "granted") {
    Alert.alert("Permission Denied", "...");
    return; // stop here if no permission
  }
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ["images"], // only show images, not videos
    allowsEditing: true, // show crop UI after selection
    aspect: [1, 1], // force square crop for profile pic
    quality: 0.8, // 80% JPEG quality (smaller file)
  });
  if (!result.canceled && result.assets[0]) {
    setProfileImage(result.assets[0].uri);
    // result.assets is an array. [0] = first selected image.
    // .uri = local file path string.
  }
};
```

```
// onboarding.tsx – takePhoto (camera)
const takePhoto = async () => {
  const { status } =
    await ImagePicker.requestCameraPermissionsAsync();
  // Camera permission is separate from media library permission.
  // Both need to be declared in app.json AND requested at runtime.
  if (status !== "granted") {
    Alert.alert("Permission Denied", "...");
    return;
  }
  const result = await ImagePicker.launchCameraAsync({
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.8,
    // No mediaTypes here – camera always produces images.
  });
  if (!result.canceled && result.assets[0]) {
    setProfileImage(result.assets[0].uri);
  }
};
```

```
// onboarding.tsx – Alert 3-option image source picker
const showImagePicker = () => {
  Alert.alert("Select Image", "Choose an option", [
    { text: "Camera", onPress: takePhoto },
    { text: "Photo Library", onPress: pickImage },
    { text: "Cancel", style: "cancel" },
  ]);
};
// Alert.alert(title, message, buttons[])
// Each button object: { text, onPress, style }
// style: "cancel" = styled differently on iOS (red text).
// On Android it looks the same as normal.
// This gives native action-sheet behaviour without a library.
// The TouchableOpacity in JSX calls showImagePicker:
// <TouchableOpacity onPress={showImagePicker}>
```

```
// onboarding.tsx – username check + handleComplete
const handleComplete = async () => {
  if (!name || !username) {
    Alert.alert("Error", "Please fill in all fields.");
    return;
  }
  if (username.length < 3) {
    Alert.alert("Error", "Username must be at least 3 characters.");
    return;
  }
  setIsLoading(true);
  try {
    const { data: existingUser } = await supabase
      .from("profiles")
      .select("id")
      .eq("username", username) // BUG: typo! "username" not "username"
      .single();
    // .from("profiles") = query the profiles table
    // .select("id") = only fetch the id column (efficiency)
    // .eq("col", val) = WHERE col = val
    // .single() = expect 0 or 1 result (not an array)
    if (existingUser) {
      Alert.alert("Error", "Username already taken.");
      setIsLoading(false);
      return;
    }
  } catch (error) { ... }
};
```

Critical typo: 'username' instead of 'username' in the Supabase query

The `.eq('username', username)` line queries a column called 'username' which does not exist. Supabase returns 0 results every time, meaning `existingUser` is always null, meaning ANYONE can register a duplicate username. TypeScript cannot catch this — it is a string value, not a type. Fix: change 'username' to 'username'. Then test with an actual duplicate.

```
// onboarding.tsx – conditional image display in JSX
{profileImage
  ? (
    <Image
      source={{ uri: profileImage }}
      style={styles.profileImage}
    />
  )
  : (
    <View style={styles.placeholderImage}>
      <Text style={styles.placeholderText}>+</Text>
    </View>
  )
}
// profileImage = null -> falsy -> shows "+" placeholder
// profileImage = "file://..." -> truthy -> shows actual Image
// This is the most common conditional rendering pattern in RN.
```

`_layout.tsx`

`app/(tabs)/_layout.tsx`

Purpose of this file

This layout defines the bottom tab bar for the main app. Each `Tabs.Screen` maps a filename to a tab with a title and icon. The `focused` prop lets you switch between filled and outline icons depending on whether that tab is currently selected.

```
// app/(tabs)/_layout.tsx – full file annotated
import { Tabs } from "expo-router";
import { Ionicons } from "@expo/vector-icons";
// @expo/vector-icons ships icon sets (Ionicons, MaterialIcons, etc.)
// Ionicons has 1300+ icons. name="home" picks the icon by string.
import { GestureHandlerRootView }
from "react-native-gesture-handler";
// Needed here because profile.tsx uses BottomSheet (gesture-based).
// GestureHandlerRootView must be an ancestor of gesture components.
export default function TabsLayout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <Tabs screenOptions={{ tabBarActiveTintColor: "crimson" }}>
        // tabBarActiveTintColor = color of the SELECTED tab icon+label.
        <Tabs.Screen
          name="index" // matches index.tsx filename
          options={{
            title: "Home",
            tabBarIcon: ({ color, size, focused }) => (
              <Ionicons
                name={focused ? "home" : "home-outline"}
                // focused = is this tab currently selected?
                // true -> filled "home" icon
                // false -> outline "home-outline" icon
                color={color} // driven by tabBarActiveTintColor
                size={size} // standard tab bar icon size (~24)
              />
            ),
          }}
        />
        // Same pattern repeats for "about" and "profile" tabs.
      </Tabs>
    </GestureHandlerRootView>
  );
}
```

Why `tabBarIcon` is a function, not a component

Expo Router passes props (color, size, focused) to the icon function at render time. By making it a function, the tab bar can control what color and size the icon renders at. If you passed a static component instead, it would always be the same color regardless of selection state.

profile.tsx

app/(tabs)/profile.tsx

Purpose of this file

The profile tab screen. It introduces three new concepts: `useRef` (hold a reference to a component without triggering re-renders), `useCallback` (memoize a function), and `BottomSheet` from `@gorhom/bottom-sheet` (a sliding panel from the bottom). These patterns appear frequently in EarGuard too.

```
// profile.tsx – useRef explained
import { useRef, useCallback } from "react";
import BottomSheet, { BottomSheetView }
from "@gorhom/bottom-sheet";
const bottomSheetRef = useRef<BottomSheet>(null);
// useRef creates a mutable container that persists across renders.
// Key difference from useState:
// - useState change -> triggers re-render (screen updates)
// - useRef change -> NO re-render (silent update)
// Use useRef when you need to hold a value or reference
// but do NOT need the screen to update when it changes.
// <BottomSheet> = TypeScript: this ref points to a BottomSheet.
// null = initial value (component not mounted yet).
// After the component mounts, React sets .current to the
// actual BottomSheet instance automatically.
```

```
// profile.tsx – useCallback and opening the sheet
const openSheet = useCallback(() => {
  bottomSheetRef.current?.expand();
  // ?. = optional chaining
  // If .current is null (not mounted), stop here – no crash.
  // If .current exists, call .expand() on the BottomSheet.
  // .expand() = BottomSheet built-in method to open/show the sheet.
}, []);
// useCallback memoizes the function.
// Without it: openSheet is recreated as a new function every render.
// With [] deps: created once and reused forever.
// Here it is a minor optimization, but important habit for EarGuard.
// In EarGuard: audio callbacks must be stable or they cause loops.
```

```
// profile.tsx – BottomSheet component props
<BottomSheet
  ref={bottomSheetRef}
  // Connects the ref. After mount, bottomSheetRef.current
  // = this BottomSheet instance. Enables openSheet() to work.
  index={-1}
  // Starting position. -1 = fully hidden (closed).
  // 0 = first snapPoint. 1 = second snapPoint, etc.
  snapPoints={["50%"]}
  // Array of heights the sheet snaps to.
  // "50%" = sheet opens to half the screen height.
  // Can have multiple: ["25%", "50%", "90%"]
  enablePanDownToClose={true}
  // User can swipe DOWN to dismiss the sheet.
  // Without this: sheet stays open until you call .close()
>
  <BottomSheetView style={styles.sheetContent}>
    <Text>Hello from the Bottom Sheet!</Text>
  </BottomSheetView>
  // BottomSheetView = BottomSheet version of View.
  // Use this inside BottomSheet instead of regular View.
</BottomSheet>
```

Why useRef instead of useState for the sheet reference

You do not want the screen to re-render just because the sheet opened or closed. `useState` would trigger a re-render on every change. `useRef` holds the reference silently. The sheet is controlled imperatively (call `.expand()` / `.close()`) rather than declaratively (set state = open).

index.tsx

app/(tabs)/index.tsx

Purpose of this file

This is the Home tab screen. Right now it is a playground from early tutorial work — it has a Hello World text, a TextInput, an ActivityIndicator, and two different ways to navigate. Not production code yet, but it is a great reference for the two navigation patterns.

```
// index.tsx – two navigation patterns side by side
import { Link, useRouter } from "expo-router";
export default function Index() {
  const router = useRouter();
  return (
    <View style={styles.container}>
      <Text style={styles.helloWorldTitle}>Hello World.</Text>
      <TextInput placeholder="Email" />
      // Note: no value or onChangeText – not wired up.
      // This is why you need useState for real forms.
      <ActivityIndicator size="large" />
      // Spinner with no condition – always spinning.
      // In real code: {isLoading && <ActivityIndicator />}
      { /* PATTERN 1: Link component (declarative) */ }
      <Link href="/about">Go to about screen</Link>
      // Renders as a pressable text link.
      // Simplest navigation method.
      { /* PATTERN 2: useRouter (imperative) */ }
      <TouchableOpacity onPress={() => router.push("/about")}>
        <Text>Navigate</Text>
      </TouchableOpacity>
      // More flexible – can add logic before navigating.
    </View>
  );
}
```

Link vs router.push — which to use when

Link is simpler and declarative — great for static nav links (like footer links). router.push is imperative — use it when you need logic before navigating (validation, auth check, loading state). In your auth screens you always use router.push/replace because navigation happens inside async handler functions, not directly in JSX.

Active Bugs Summary

These bugs exist in your CURRENT uploaded files right now. Fix these before moving forward in the tutorial.

signup.tsx Zombie useEffect navigates to onboarding on mount

Fix: Delete useEffect lines 22-24. Put router.replace() inside handleSignUp after await signUp().

signup.tsx Password minimum is 3 — Supabase requires 6

Fix: Change password.length < 3 to password.length < 6.

login.tsx TextInput from wrong package (gesture-handler)

Fix: Change import to from 'react-native' not 'react-native-gesture-handler'.

login.tsx Sign In button has no onPress

Fix: Add onPress={handleSignIn} to the TouchableOpacity.

login.tsx No useState for email/password

Fix: Add const [email, setEmail] = useState("") and same for password.

onboarding.tsx Typo: 'username' in Supabase query

Fix: Change .eq('username', username) to .eq('username', username).

AuthContext.tsx setUser() never called after signIn/signUp

Fix: Map data.user to your User interface and call setUser() after successful auth.

_layout.tsx isAuthenticated = false hardcoded

Fix: Replace with const { user } = useAuth() and change if (!isAuth) to if (!user).

What comes next — Part 3

Part 3 covers every bug in detail with root cause analysis, the 10 prevention laws, and a complete quick reference cheatsheet. It is the debugging companion to this code walkthrough.

Spanish word of the day: el archivo

archivo (ar-CHEE-voh) = the file. Example: El archivo tiene un error. (The file has an error.) You just walked through 10 archivos. You earned this one.